

ВМА – Задание №2

Отчет

1) Персонально подготовить ваши работающие программы, реализующие решение СЛАУ $Ax=b$ с матрицей Гильберта размерности n методом Гаусса «единственного деления». Вектор свободных членов системы вычислить по формуле $b=Ax^*$, где точное значение вектора решения $x^* = (1, 1, \dots, 1)^T$, а размерность системы n – задаётся произвольно.

Код матрицы Гильберта:

```
public class HilbertMatrix {
    private final int size;
    private final double[][] matrix;

    public HilbertMatrix(int n) {
        size = n;
        if (size <= 0) {
            throw new IllegalArgumentException("Size must be positive, got: "
+ size);
        }
        matrix = new double[size][size];

        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                matrix[i][j] = 1.0 / (i + j + 1);
            }
        }
    }

    public double[][] getMatrix() {
        double[][] copy = new double[size][size];
        for (int i = 0; i < size; i++) {
            copy[i] = matrix[i].clone();
        }
        return copy;
    }

    public int getSize() {
        return size;
    }
}
```

Код основного цикла:

```
private static void forwardStroke(double[][] matrix, double[] rhs, int i,
double eps) {
    int size = matrix.length;
    double aii = matrix[i][i];
    if (Math.abs(aii) < eps) {
        throw new ArithmeticException("Matrix is singular or nearly singular
at column " + i);
    }

    for (int j = i; j < size; j++) {
        matrix[i][j] /= aii;
        if (Double.isInfinite(matrix[i][j]) || Double.isNaN(matrix[i][j])) {
            throw new ArithmeticException("Division by zero");
        }
    }
}
```

```

    }
    rhs[i] /= aii;
    if (Double.isInfinite(rhs[i]) || Double.isNaN(rhs[i])) {
        throw new ArithmeticException("Division by zero");
    }

    for (int k = i + 1; k < size; k++) {
        double aki = matrix[k][i];
        for (int t = i; t < size; t++) {
            matrix[k][t] -= (matrix[i][t] * aki);
        }
        rhs[k] -= (rhs[i] * aki);
    }
}

private static void backStroke(double[][] matrix, double[] rhs, double eps) {
    int size = matrix.length;
    for (int i = size - 1; i >= 0; i--) {
        for (int k = i - 1; k >= 0; k--) {
            rhs[k] -= (rhs[i] * matrix[k][i]);
            matrix[k][i] = 0;
        }
    }
}

public static double[] solve(double[][] matrix, double[] rhs, double eps) {
    validate(matrix, rhs);

    int size = matrix.length;
    double[][] a = new double[size][size];
    for (int i = 0; i < size; i++) {
        a[i] = matrix[i].clone();
    }
    double[] x = rhs.clone();

    for (int i = 0; i < size; i++) {
        forwardStroke(a, x, i, eps);
    }

    backStroke(a, x, eps);
    return x;
}

```

Вывод этой части кода:

Enter matrix size: 3

Enter 3 x: 1 2 3

Hilbert matrix:

1,000000e+00	5,000000e-01	3,333333e-01
5,000000e-01	3,333333e-01	2,500000e-01
3,333333e-01	2,500000e-01	2,000000e-01

Task result:	1,000000000e+00	2,000000000e+00	3,000000000e+00
--------------	-----------------	-----------------	-----------------

My default solve result:	1,000000000e+00	2,000000000e+00	3,000000000e+00
--------------------------	-----------------	-----------------	-----------------

Compare solve result:	equal	equal	equal
-----------------------	-------	-------	-------

2) По найденным программой решениям СЛАУ (для $n=3,7,11,15,19$) запрограммировать и вычислить нормы (кубическую, октаэдрическую, сферическую) полученных векторов ошибки $err_i = \|x - x^*\|_i$.

Код для нахождения норм:

```
public static double[] vectorsDifference(double[] mas1, double[] mas2) {
    if (mas1 == null || mas2 == null) {
        throw new IllegalArgumentException("X or X* must not be null");
    }

    int n = mas1.length;
    if (n == 0) {
        throw new IllegalArgumentException("Matrix must not be empty");
    }

    if (mas2.length != n) {
        throw new IllegalArgumentException("RSH and matrix sizes must be equal");
    }

    double[] err = new double[n];

    for (int i = 0; i < n; i++) {
        err[i] = mas1[i] - mas2[i];
    }
    return err;
}

public static double cubicNorm(double[] mas) {
    double maxAbs = mas[0];
    for (double ma : mas) {
        if (maxAbs < Math.abs(ma)) {
            maxAbs = Math.abs(ma);
        }
    }
    return maxAbs;
}

public static double octahedralNorm(double[] mas) {
    double sum = 0;
    for (double ma : mas) {
        sum += Math.abs(ma);
    }
    return sum;
}

public static double sphericalNorm(double[] mas) {
    double sum = 0;
    for (double ma : mas) {
        sum += ma * ma;
    }
    return Math.sqrt(sum);
}
```

3) Результаты выполнения программы свести в таблицу $[n \setminus err_i]$ и проанализировать полученные результаты.

4) На основе отлаженной программы схемы единственного деления подготовить две её модификации, реализующие методы Гаусса с выбором главного элемента по столбцу и по строке. Полученные результаты выполнения программ свести в таблицы по аналогии с пунктом 3).

```
public static double[] columnSolve(double[][] matrix, double[] rhs, double
eps) {
    validate(matrix, rhs);

    int size = matrix.length;
    double[][] a = new double[size][size];
    for (int i = 0; i < size; i++) {
        a[i] = matrix[i].clone();
    }
    double[] x = rhs.clone();

    int pivotRow;
    double maxAbs;

    for (int i = 0; i < size; i++) {
        pivotRow = i;
        maxAbs = Math.abs(a[pivotRow][pivotRow]);

        for(int c = i + 1; c < size; c++) {
            if (Math.abs(a[c][i]) > maxAbs) {
                pivotRow = c;
                maxAbs = Math.abs(a[c][i]);
            }
        }

        if (pivotRow != i) {
            double[] bufferStr = a[i].clone();
            a[i] = a[pivotRow];
            a[pivotRow] = bufferStr;

            double bufferX = x[i];
            x[i] = x[pivotRow];
            x[pivotRow] = bufferX;
        }

        forwardStroke(a, x, i, eps);
    }

    backStroke(a, x, eps);
    return x;
}

public static double[] rowSolve(double[][] matrix, double[] rhs, double eps)
{
    validate(matrix, rhs);

    int size = matrix.length;
    double[][] a = new double[size][size];
    for (int i = 0; i < size; i++) {
        a[i] = matrix[i].clone();
    }
    double[] x = rhs.clone();

    int pivotColumn;
    double maxAbs;
    int[] perVec = new int[size];
    for (int i = 0; i < size; i++) {
```

```

        perVec[i] = i;
    }

    for (int i = 0; i < size; i++) {
        pivotColumn = i;
        maxAbs = Math.abs(a[pivotColumn][pivotColumn]);

        for (int c = i + 1; c < size; c++) {
            if (Math.abs(a[i][c]) > maxAbs) {
                pivotColumn = c;
                maxAbs = Math.abs(a[i][c]);
            }
        }

        if (pivotColumn != i) {
            double tempElem;
            int tempInd;
            for (int j = 0; j < size; j++) {
                tempElem = a[j][i];
                a[j][i] = a[j][pivotColumn];
                a[j][pivotColumn] = tempElem;
            }
            tempInd = perVec[i];
            perVec[i] = perVec[pivotColumn];
            perVec[pivotColumn] = tempInd;
        }

        forwardStroke(a, x, i, eps);
    }

    backStroke(a, x, eps);

    double tempX;
    int tempPer;
    for (int i = 0; i < size; i++) {
        if (perVec[i] != i) {
            tempX = x[i];
            x[i] = x[perVec[i]];
            x[perVec[i]] = tempX;

            tempPer = perVec[perVec[i]];
            perVec[perVec[i]] = perVec[i];
            perVec[i] = tempPer;
            i--;
            continue;
        }
    }

    return x;
}

```

3

	Cubic Norm	Octahedral Norm	Spherical Norm
Default solve:	7,54951657e-15	1,57651669e-14	1,04266434e-14
Column solve:	1,25455202e-14	2,68673972e-14	1,76347316e-14
Row solve:	1,02140518e-14	2,19824159e-14	1,43986981e-14

7

	Cubic Norm	Octahedral Norm	Spherical Norm
Default solve:	2,53114991e-08	7,21045835e-08	3,70768809e-08
Column solve:	2,24577184e-08	6,39667204e-08	3,28821151e-08
Row solve:	1,96606594e-08	5,59895563e-08	2,87740970e-08

11

	Cubic Norm	Octahedral Norm	Spherical Norm
Default solve:	1,43868194e-02	4,99807671e-02	2,27690727e-02
Column solve:	1,31960695e-02	4,58522180e-02	2,08837812e-02
Row solve:	1,28148767e-02	4,45164115e-02	2,02818392e-02

15

	Cubic Norm	Octahedral Norm	Spherical Norm
Default solve:	7,40261180e+00	3,23643689e+01	1,26626849e+01
Column solve:	1,08587072e+01	4,48399105e+01	1,80097345e+01
Row solve:	7,24036742e+00	2,56061814e+01	1,13636841e+01

19

	Cubic Norm	Octahedral Norm	Spherical Norm
Default solve:	4,61489487e+01	2,45369386e+02	8,40511500e+01
Column solve:	2,71427961e+01	1,33496094e+02	4,66024354e+01
Row solve:	1,29898148e+01	7,66538054e+01	2,55659632e+01

Король Д.Е.
Группа №8А